



UNIVERSIDAD AUTÓNOMA DE AGUASCALIENTES

Reporte: Proyecto Final

Alumnos:

Diego Joao Sánchez López ID - 341105

Joshua Alejandro Bravo González ID – 549795

Erich Leonardo Castruita Rodríguez ID - 340804

Rodrigo Vázquez Zuñiga ID - 549632

Semestre: 4-A

Centro de ciencias básicas

Carrera: Ingeniería en sistemas computacionales

Materia: UNIX

Maestro: Guillermo Dominguez Aguilar

1.- ABC en Script Shell	3
1. 1-Introducción	3
1. 2-Objetivo	3
1. 3-Desarrollo	3
2.- Proyecto en lenguaje de alto nivel	6
2. 1-Introducción	6
2. 2-Objetivo	6
3. 3-Desarrollo	7
Conclusión	
Bibliografía	

1.- ABC en Script Shell

1. 1-Introducción

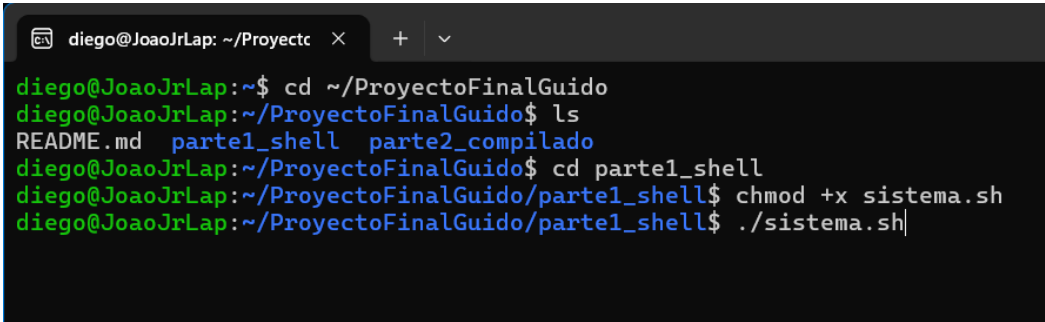
La línea de comandos de UNIX y GNU/Linux es algo que la mayoría de las personas pasan por alto o no le dan su respectiva importancia, sin embargo, ofrece herramientas útiles para automatizar tareas y administrar el sistema. Una de sus ventajas es que permite trabajar con archivos de texto de forma directa mediante Shell Scripting. En esta primera parte del proyecto implementamos un sistema de gestión de alumnos y calificaciones. Con esto buscamos explorar cómo los scripts manejan información de manera estructurada, usando comandos estándar y estructuras de control para simular una base de datos con altas, bajas, cambios y reportes.

1. 2-Objetivo

El objetivo de esta parte del proyecto es desarrollar e implementar un sistema interactivo usando Script Shell, debe permitir gestionar de forma eficiente la información de alumnos y sus calificaciones mediante un archivo de datos estructurado, además, deberá ejecutar operaciones ABC y modificaciones, también deberá de generar reportes automatizados que faciliten el análisis de la información almacenado. De esta manera, se busca aplicar los conocimientos adquiridos sobre programación en Shell, manipulación de archivos y administración de datos en entornos UNIX.

1. 3-Desarrollo

Esta es la solución desarrollada para cumplir con nuestro objetivo, el resultado nos permite almacenar y manipular información de manera sencilla utilizando archivos de texto, aprovechando las capacidades de procesamiento y automatización que ofrece la terminal de Linux.

A screenshot of a terminal window with a dark background. The window title bar shows 'diego@JoaoJrLap: ~/Proyecto' and standard window controls. The terminal displays a series of commands and their outputs: the user navigates to the directory ~/ProyectoFinalGuido, lists the contents (showing README.md, parte1_shell, and parte2_compilado), navigates into the parte1_shell directory, sets permissions for sistema.sh, and finally executes the script. The prompt changes to reflect the current directory at each step.

```
diego@JoaoJrLap:~$ cd ~/ProyectoFinalGuido
diego@JoaoJrLap:~/ProyectoFinalGuido$ ls
README.md  parte1_shell  parte2_compilado
diego@JoaoJrLap:~/ProyectoFinalGuido$ cd parte1_shell
diego@JoaoJrLap:~/ProyectoFinalGuido/parte1_shell$ chmod +x sistema.sh
diego@JoaoJrLap:~/ProyectoFinalGuido/parte1_shell$ ./sistema.sh
```

La información se almacena en un archivo de texto llamado "alumnos.txt", que funciona como una base de datos simple. Cada línea representa un alumno y contiene los campos de matrícula, nombre, apellidos, semestre y calificación final. Así la estructura mantiene un formato uniforme para todas las operaciones del sistema.

```
diego@JoaoJrLap: ~/Proyecto
=====
          SISTEMA GESTOR DE ALUMNOS
=====
1. Alta de Alumno
2. Baja de Alumno
3. Consulta de Alumno
4. Modificación de Calificación
5. Reporte: Alumnos Aprobados
6. Reporte: Promedio General
7. Reporte: Alumnos por Semestre
8. Salir
=====
Seleccione una opción:
```

En la estructura general del sistema, se implementó un script denominado “sistema.sh”, el cual presenta un menú interactivo que permite al usuario acceder a las diferentes funciones disponibles, con este menú facilitamos la interacción con el archivo de datos sin necesidad de manipularlo directamente ya que, el programa permanece en ejecución hasta que el usuario decide finalizar la sesión, proporcionando una experiencia de uso similar a la de un sistema de administración básico. El menú por sí solo no haría nada, así que implementamos varias funciones. La de alta registra nuevos alumnos, verificando antes que la matrícula no exista para evitar duplicados. La de baja elimina registros buscando por matrícula. La consulta localiza y muestra los datos de un alumno específico, y la modificación permite actualizar la calificación final de un registro ya creado.

```
diego@JoaoJrLap: ~/Proyecto
=== ALTA DE ALUMNO ===
Ingrese Matrícula: 3333
Nombre: Pavel
Apellidos: Sanchez
Semestre: 2
Calificación Final: 8
Alumno registrado exitosamente.

Presione [Enter] para continuar...
```

```
diego@JoaoJrLap: ~/Proyectc x + v
=== CONSULTA DE ALUMNO ===
1. Buscar por Matrícula
2. Mostrar todos los alumnos
Opción: 1
Ingrese Matrícula: 3333

Matrícula,Nombre,Apellidos,Semestre,Calificación
3333,Pavel,Sanchez,2,8

Presione [Enter] para continuar...
```

Estas operaciones forman la base del sistema de administración y permiten mantener actualizada la información académica almacenada.

Aparte de la administración, el sistema genera tres reportes. El primero muestra la lista de alumnos aprobados según el criterio de calificación. El segundo calcula el promedio general de las calificaciones registradas. El tercero cuenta cuántos alumnos hay en cada semestre para conocer su distribución. Con estos reportes el sistema presenta la información de forma ordenada.

```
diego@JoaoJrLap: ~/Proyectc x + v
=== REPORTE 1: ALUMNOS APROBADOS (>= 70) ===
Matrícula,Nombre,Apellidos,Semestre,Calificación
123456,Leonardo,Perez,4,85
234567,Ana,Gomez,4,90
456789,Maria,Martinez,5,95

Presione [Enter] para continuar...
```

```
diego@JoaoJrLap: ~/Proyectc x + v
=== REPORTE 2: PROMEDIO GENERAL ===
Total de Alumnos: 6
Promedio General: 66.33

Presione [Enter] para continuar...
```

```
diego@JoaoJrLap: ~/Proyectc x + v
=== REPORTE 3: CONTEO DE ALUMNOS POR SEMESTRE ===
Semestre | Cantidad
2         | 1
4         | 2
5         | 2
6         | 1

Presione [Enter] para continuar...
```

El desarrollo se hizo en Shell Scripting (Bash), usando las herramientas nativas de UNIX para el manejo de archivos y texto. Para la lógica del programa empleamos estructuras de control como condicionales, ciclos y cases, además de comandos para buscar, leer y modificar la información en los archivos.

Parte 2: Proyecto en Lenguaje de Alto Nivel

Introducción

Para la segunda parte del proyecto, se nos solicitó implementar un programa en un lenguaje de alto nivel compilado y automatizar su proceso de compilación mediante el uso de la herramienta make. Con el objetivo de presentar una propuesta creativa e interactiva, el equipo decidió utilizar el clásico juego de "Snake" (La Víbora) ejecutado completamente desde la línea de comandos.

El código fuente principal está escrito en lenguaje C y hace uso de la librería estándar de UNIX ncurses, la cual permite crear interfaces de usuario y manejar eventos de teclado directamente en la terminal, alejándose de los programas secuenciales tradicionales. Para cumplir con el propósito didáctico de la materia, se obtuvo una versión de código abierto del juego y el esfuerzo del equipo se centró en analizar su estructura, adaptarlo a nuestro entorno y, lo más importante, diseñar un archivo Makefile robusto que organice su compilación y limpieza de manera profesional.

```
diego@JoaoJrLap:~/ProyectoFinalGuido/parte2_compilado$ make
Compilando snake...
gcc -Wall -Wextra -o snake snake.c -lncurses
diego@JoaoJrLap:~/ProyectoFinalGuido/parte2_compilado$ |
```

Objetivo

El objetivo principal de esta segunda etapa fue comprender y aplicar el proceso de construcción y automatización de software en lenguajes de alto nivel dentro de un entorno UNIX, específicamente, se buscó implementar un programa en lenguaje C interactuando con la consola a través de la librería ncurses, y orquestar todo su proceso de compilación mediante un archivo Makefile.

Con esto, se pretendió demostrar la capacidad de gestionar dependencias, limpiar el entorno de trabajo y automatizar tareas repetitivas de compilación, habilidades fundamentales para la administración eficiente de proyectos de software en sistemas Linux.

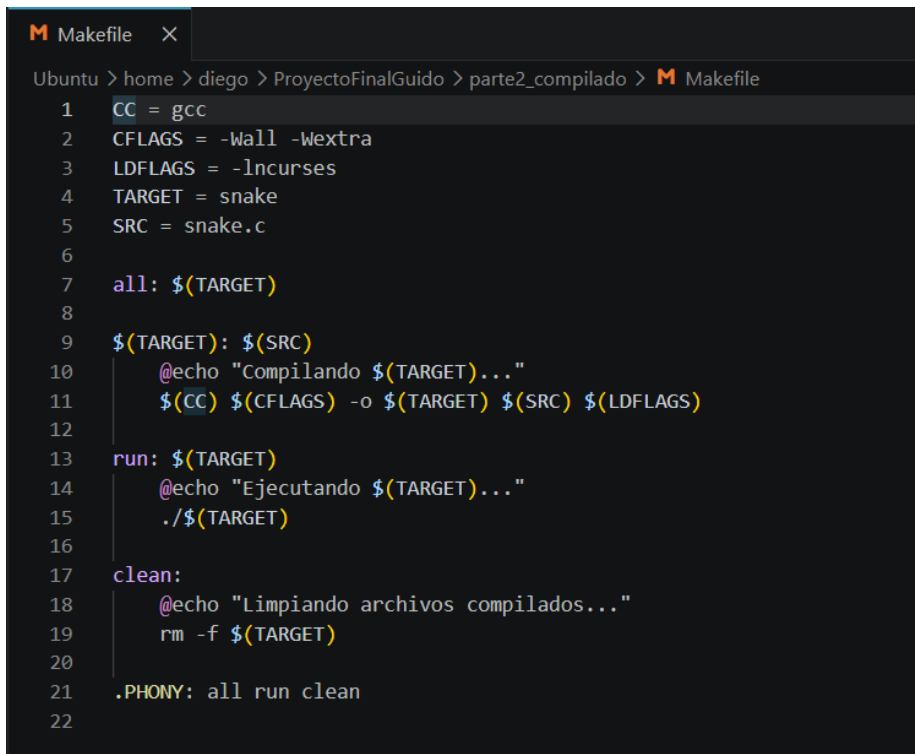
Desarrollo

El desarrollo de esta fase se dividió en dos componentes principales: el código fuente (snake.c) y su archivo de automatización (Makefile).

1. El Archivo Makefile

El corazón de esta segunda parte es el Makefile. En lugar de invocar manualmente al compilador de GNU (gcc) con todas sus banderas y librerías cada vez que se requiere probar el juego, se configuraron reglas específicas:

- **Variables:** Se definieron variables como CC = gcc y LDFLAGS = -lncurses para mantener el archivo limpio y escalable.
- **Regla all:** Es la regla por defecto que toma el archivo snake.c y genera el ejecutable final llamado snake.
- **Regla run:** Permite iniciar el juego inmediatamente después de compilarlo.
- **Regla clean:** Borra los archivos binarios generados para mantener el directorio de trabajo limpio.

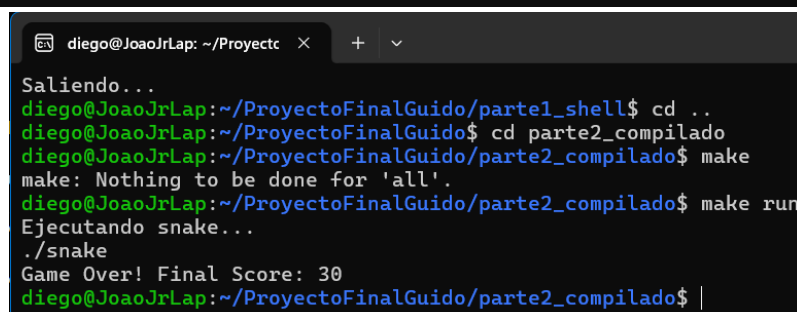


```
M Makefile X
Ubuntu > home > diego > ProyectoFinalGuido > parte2_compilado > M Makefile
1 CC = gcc
2 CFLAGS = -Wall -Wextra
3 LDFLAGS = -lncurses
4 TARGET = snake
5 SRC = snake.c
6
7 all: $(TARGET)
8
9 $(TARGET): $(SRC)
10 @echo "Compilando $(TARGET)..."
11 $(CC) $(CFLAGS) -o $(TARGET) $(SRC) $(LDFLAGS)
12
13 run: $(TARGET)
14 @echo "Ejecutando $(TARGET)..."
15 ./$(TARGET)
16
17 clean:
18 @echo "Limpiando archivos compilados..."
19 rm -f $(TARGET)
20
21 .PHONY: all run clean
22
```

Al ejecutar el comando make en la terminal, el sistema lee estas instrucciones, verifica si hay cambios en el código fuente, y procede a enlazar la librería ncurses de manera automática, mostrando mensajes en español en la consola para informar al usuario sobre el progreso.

2. Ejecución del Juego (snake.c)

El programa funciona mediante un bucle infinito que se ejecuta hasta que el jugador pierde. Utiliza un plano cartesiano invisible basado en las filas y columnas de la terminal (LINES y COLS). La lógica principal evalúa en tiempo real las colisiones de la "cabeza" de la serpiente contra las paredes o su propio cuerpo, así como la generación aleatoria de la "fruta" que incrementa la puntuación.



Conclusión

- General:

El desarrollo de este proyecto demostró la utilidad práctica de la línea de comandos en sistemas UNIX y GNU/Linux, utilizándola no solo para ejecutar instrucciones aisladas, sino como un entorno de trabajo completo. A través de las dos prácticas realizadas, se aplicaron de manera directa conceptos de administración de información y automatización en el desarrollo de software.

En la primera parte, la creación del sistema de gestión con Bash permitió observar cómo las herramientas nativas del sistema operativo pueden manejar registros de forma estructurada. Mediante el uso de scripts, se logró programar un sistema funcional de altas, bajas, modificaciones y

consultas sobre un archivo de texto plano. Esto comprueba que la terminal es una plataforma capaz de procesar datos y generar reportes sin necesidad de instalar gestores de bases de datos adicionales.

Durante la segunda fase, la implementación del programa en lenguaje C con la librería ncurses sirvió para trabajar con interfaces gráficas básicas y eventos de teclado directamente en la consola. Sin embargo, el punto clave de esta etapa fue la configuración del archivo Makefile. Al definir reglas específicas de compilación y limpieza, se eliminó la necesidad de escribir comandos largos de manera repetitiva, agilizando las pruebas y el control del código fuente.

Para finalizar, la realización de ambas partes del proyecto refuerza los conocimientos técnicos necesarios para trabajar de manera eficiente en sistemas basados en Linux. Se logró aplicar desde la manipulación de flujos de texto y la programación de menús interactivos, hasta la automatización del proceso de compilación para programas de alto nivel, cumpliendo satisfactoriamente con los objetivos planteados.

- **Individuales**

Rodrigo: En lo personal, este proyecto me hizo valorar la terminal de Linux, una herramienta que solía pasar por alto por la costumbre de usar entornos gráficos. Con la primera parte en Bash, comprobé que es posible administrar datos y automatizar tareas de forma muy eficiente sin depender de programas pesados o bases de datos complejas, usando únicamente los comandos del sistema. Por otro lado, la fase con C y la librería ncurses fue un buen reto para entender la interacción directa con la consola. Sin embargo, mi mayor aprendizaje fue la configuración del Makefile; automatizar el proceso de compilación me ahorrará mucho tiempo y evitará errores manuales en mis futuros trabajos. En resumen, me llevo conocimientos muy prácticos. Ahora veo la línea de comandos como una herramienta indispensable y de uso diario, la cual me brinda un control total sobre el código que desarrollo y la administración de los sistemas.

Joshua: La verdad este proyecto me sirvió para cambiar la idea que tenía de la terminal de Linux, porque antes la veía nomás como un lugar para escribir comandos sueltos y ya, no como algo donde de verdad puedes construir cosas funcionales. Con la primera parte en Bash me di cuenta de que sí se puede armar un sistema completo de altas, bajas y consultas usando puro archivo de texto, sin necesidad de meterle una base de datos pesada. El manejo de los reportes fue lo que más se me

hizo interesante porque ahí vi cómo la terminal puede ordenar y presentar la información de forma clara con puras herramientas del sistema. Ya en la parte de C con ncurses se me complicó un poco más entender cómo manejar los eventos del teclado y las colisiones en tiempo real, pero al final valió la pena por lo que aprendí. Aún así, lo que más me dejó fue el Makefile, porque automatizar la compilación me va a ahorrar un montón de tiempo y me evita andar escribiendo el mismo comando largo cada rato. En general me llevo bastante de este proyecto. Ahora veo la línea de comandos como una herramienta que sí vale la pena dominar, porque te da mucho control tanto sobre el código como sobre la administración del sistema.

Joao: Este proyecto me permitió poner en práctica varios de los comandos y funciones de la terminal UNIX, así como la programación en Script Shell y la administración de archivos, aunque algunas actividades representaron un reto al principio, sobre todo porque no tenía el conocimiento completo de lo que, hacía cada comando, así que el proceso me ayudó a comprender mejor cómo funcionan las herramientas del sistema operativo y cómo pueden utilizarse para automatizar tareas de manera eficiente. El desarrollo del sistema de gestión de alumnos me permitió observar que, por más complicado o raro que parezca la terminal, se puede utilizar para crear programas capaces de administrar información de forma organizada. Por otro lado, la compilación y ejecución del proyecto en lenguaje C mediante Makefile me ayudó a entender el proceso de construcción de programas dentro de un entorno UNIX. De manera personal, este proyecto me sirvió para darme cuenta de que el aprender cómo funciona la terminal de GNU/Linux y UNIX es una habilidad muy útil hoy en día, es sin duda, una habilidad muy importante que debo de seguir mejorando y reforzando, debido a que es muy utilizada en casi todos los ámbitos de la carrera hoy en día.

Leonardo: Personalmente considero que el mayor valor de este proyecto fue entender el ciclo de desarrollo completo que ofrece UNIX, contrastando dos paradigmas de programación muy distintos, por una parte la creación del script en Bash me demostró la inmediatez y flexibilidad que tiene la terminal para solucionar problemas de bases de datos y filtrado de texto usando comandos crudos, y por otra parte, la fase del juego en C me enseñó el lado estructurado y compilado del sistema operativo. Creo que lo que más puedo destacar de esta última etapa fue la integración del Makefile; el hecho de poder tomar un código fuente, adaptarlo y construir una especie de libro de recetas que le diga a la computadora exactamente cómo compilarlo, ejecutarlo y limpiarlo con una

sola palabra, es algo que cambia por completo la forma en la que considerare abordar mis proyectos futuros. Con todo esto, puedo decir que dominar Linux no es solo aprender comandos de memoria, sino saber combinar sus herramientas para crear entornos de trabajo ágiles y profesionales.

Bibliografía

Parte 1 (Scripting en Bash, Awk, Sed, Grep)

- Free Software Foundation. (2024). *Bash reference manual*. GNU Operating System. <https://www.gnu.org/software/bash/manual/bash.html>
- Free Software Foundation. (2023). *sed, a stream editor*. GNU Operating System. <https://www.gnu.org/software/sed/manual/sed.html>
- Free Software Foundation. (2024). *GAWK: Effective AWK programming*. GNU Operating System. <https://www.gnu.org/software/gawk/manual/gawk.html>

Parte 2 (Makefile, Lenguaje C y ncurses)

- Free Software Foundation. (2023). *GNU make manual*. GNU Operating System. <https://www.gnu.org/software/make/manual/make.html>
- Dickey, T. E. (2024). *NCURSES programming HOWTO*. The Linux Documentation Project. [Inserta el enlace aquí]